

Documentation:

Shell interface in Zephyr RTOS and ADC streaming application using DMA

Lakshmeesh K P

Note: This documentation is based on Zephyr RTOS version 4.2.0

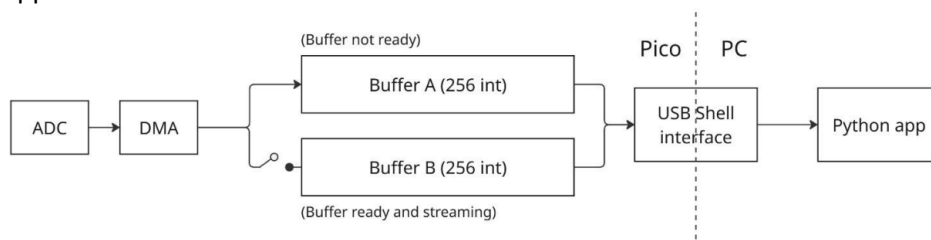
Reference: [\[1\]](#)

Motivation:

- Streaming from pico (using DMA) to PC serially and having a shell interface to control actions, may be useful for many real-time applications.
For example,
 - o Keyword detection and actuating some action serially, or storing the recorded audio on PC for later use.
 - o Speech enhancement on pico and transfer the clean audio to the PC for storing.
- I started building a demo application which streams ADC data from a microphone to PC over a shell interface, where we can control the recording to start/stop.

App explanation:

- Application flow:



- Setting 2 virtual COM ports:
 - o I setup 2 COM ports, for convenience of debugging. One for shell commands and also receiving printk statements for debugging. Second for raw data streaming. A python script was made to manage both COM ports from PC.
 - o I use the zephyr snippet “cdc-acm-console” while building which creates a COM port. To create another port, we only need to define another node in the .overlay file, with the required compatible label.
- Note regarding Baud-rate:
 - o the CDC-ACM ignores baud rate settings and transfers data at the full speed for USB2.0 (ie 12Mbps). The baud rate settings mentioned in the zephyr drivers are only for legacy compatibility reasons, required during USB enumeration.
 - o For 16KHz, 16-bit samples, we need atleast 320Kbps ($16000 * 16 * (10/8)$), which is well under the limit. The next standard baud rate 460800 was chosen

- We register the root shell commands using this macro:

```
SHELL_CMD_REGISTER(start, NULL, "Start recording", cmd_start);
SHELL_CMD_REGISTER(stop, NULL, "Stop recording", cmd_stop);
SHELL_CMD_REGISTER(test_usb, NULL, "Test usb streaming", test_usb_cmd);
```

I have set the sub-command arg to NULL, but can be used to add further sub-commands to a root command.

We enter the command specified in the first arg to do instructions defined in function at 4th arg. 3rd arg is a help string.

- Define the actions to be done by the shell commands:

```
static int cmd_start(const struct shell *shell, size_t argc, char **argv){
    ARG_UNUSED(argc);
    ARG_UNUSED(argv);
    ARG_UNUSED(shell);
    // shell_print(shell, "Recording started");
    printk("Recording started\n");
    record = true;
    dma_setup2();
    dma_start(dma_dev, dma_channel);
    adc_run(true);
    return 0;
}

static int cmd_stop(const struct shell *shell, size_t argc, char **argv){
    ARG_UNUSED(argc);
    ARG_UNUSED(argv);
    ARG_UNUSED(shell);
    // shell_print(shell, "Recording stopped");
    printk("Recording stopped\n");
    adc_run(false);
    dma_stop(dma_dev, dma_channel);

    record = false;
    buffer_a.filled = false;
    buffer_a.ready = false;
    buffer_b.ready = false;

    return 0;
}
```

- The `uart_poll_out()` API is used to transmit data from buffer via the console.

```
void usb_send(const struct device *dev, uint8_t *data, size_t len){
    for (size_t i = 0; i < len; i++) {
        uart_poll_out(dev, data[i]);
    }
}
```

This api is blocking in nature, and busy waits for the duration of transmission.

Another API is `uart_fifo_fill()` which is interrupt driven. It fills the fifo as much as possible, and returns back to the CPU even if data hasn't been transmitted. So we ourselves need to keep track of bits sent so far. `uart_poll_out()` was used for safety and testing.

- We run a python script on the PC to send commands to Pico serially via the dedicated COM port and run commands.

The audio data is streamed to the PC and the script converts it into a listenable .wav file.

Results and diagnostics:

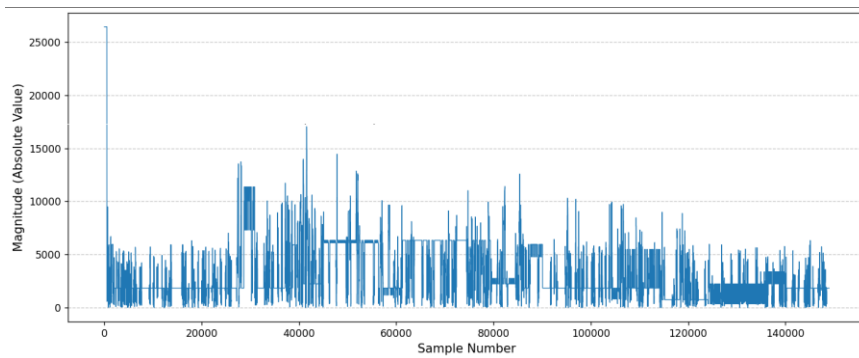
- Problem 1: The speech input is currently not comprehensible; we can only hear deep rumbles.
Problem 2: The recorded file seems to be shorter than the actually recorded duration.
To identify the root cause, the pipeline was split and checked.
- Check 1: checked if there is any issue in the USB serial connection, ie, the data transfer between pico and PC is correct. DMA/ADC were disabled, and a pre-defined array was transferred.

```
Enter command (start / stop / exit): [PICO LOG]: test_usb
[PICO LOG]: started test USB streaming (no adc or DMA used)
TEST READ: 10 bytes: b'\x01\x00\x02\x00\x03\x00\x04\x00\x05\x00'...
```

It means that there's no fault from the serial connection, and no corruptions from having a high baud rate either.

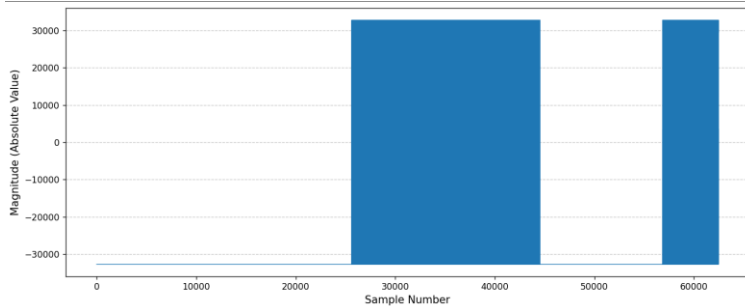
- Check 2: turned on DMA and ADC and plotted data to check any anomalies.

[illegible]

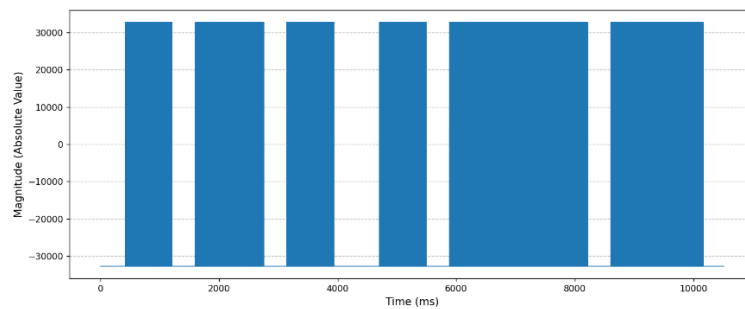


Note: the “and = entries in the hexadecimal are not an issue. They are just hexadecimals represented as a printable ASCII character. No trends or anomalies are really visible. Just wanted to check if it was looping on the same buffer or something.

- Check 3: transfer a predictable audio (like a sine wave beep), and observe



(amplitude vs sample no.)(output-1)



(amplitude vs time)(output-2)

Beep was audible, however: 1) there were gaps 2) sometimes the beep pitch felt deeper

- The pitch shift might hint some bytes getting lost periodically, causing the audio to seem deeper than expected.
- also in “output-1”, the total recorded duration was ~10s, but the .wav duration was ~4s (~40% length) for “output-2”, the total recorded duration was ~25s, but as seen in graph, duration was ~10s (again 40%, coincidence?)

Again as before, hinting some bytes getting lost.

(the 40% might as well be a coincidence, since, these two are the only test graphs which I had saved; I hadn’t checked the relation with the other test graphs during the BTP;

However, it is for sure that there exists a problem of data loss)

Possible debugs:

- (see drivers/uart.h) `uart_configure()` can be used to set the flow control. That might ensure that packets don’t drop.
- Also, `uart_poll_out()` sends a packet only if the transmitter is not full, and blocks till then (ie, above debug may not work bcos of that). So, another option to try may be to increase the transmitter buffer size itself. It can be done by increasing this kconfig option “`CONFIG_USB_CDC_ACM_RINGBUF_SIZE`” (1024 by default).